

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

DEPARTAMENTO DE COMPUTAÇÃO E TECNOLOGIA

DCT1106 - PROGRAMAÇÃO

Bacharelado em Sistemas da Informação (BEC)

Período Letivo: 2026.2

Professor: Thommas K. S. Flores

1 Resumo da Sintaxe

1.1 Leitura e Escrita de Dados

A entrada e saída padrão em C são feitas principalmente com as funções `scanf()` e `printf()`. A tabela abaixo resume os especificadores de formato mais comuns para os tipos básicos.

Tipo	Especificador (<code>printf</code>)	Especificador (<code>scanf</code>)
<code>int</code>	<code>%d</code> ou <code>%i</code>	<code>%d</code> ou <code>%i</code>
<code>float</code>	<code>%f</code>	<code>%f</code>
<code>double</code>	<code>%f</code> ou <code>%lf</code>	<code>%lf</code>
<code>char</code>	<code>%c</code>	<code>%c</code>
<code>char[]</code> (string)	<code>%s</code>	<code>%s</code>
<code>unsigned int</code>	<code>%u</code>	<code>%u</code>
<code>long int</code>	<code>%ld</code>	<code>%ld</code>
<code>unsigned long</code>	<code>%lu</code>	<code>%lu</code>
<code>short int</code>	<code>%hd</code>	<code>%hd</code>
hexadecimal	<code>%x</code> / <code>%X</code>	<code>%x</code>
octal	<code>%o</code>	<code>%o</code>

Table 1: Especificadores de formato para `printf` e `scanf`.

Observações importantes:

- No `scanf`, sempre preceda as variáveis com `&` (exceto strings, que já são endereços).
- O `printf` não requer `&`.
- Para ler vários valores, separe os especificadores por espaços ou vírgulas, conforme o formato de entrada.

1.1.1 Exemplo de Programa Base

A seguir, um programa completo que utiliza `printf` e `scanf` para ler dois números inteiros e exibir sua soma.

```
1 #include <stdio.h>
2
3 int main() {
```

```

4   int a, b, soma;
5
6   // Solicita o primeiro número
7   printf("Digite o primeiro numero inteiro: ");
8   scanf("%d", &a);
9
10  // Solicita o segundo número
11  printf("Digite o segundo numero inteiro: ");
12  scanf("%d", &b);
13
14  soma = a + b;
15
16  printf("A soma de %d e %d eh %d.\n", a, b, soma);
17
18  return 0;
19 }

```

1.2 Estruturas Condicionais

1.2.1 *if...else*

A estrutura `if` avalia uma condição e executa um bloco se a condição for verdadeira. O `else` (opcional) executa outro bloco se a condição for falsa.

```

1  if (condicao) {
2      // codigo se verdadeiro
3  } else {
4      // codigo se falso
5  }

```

1.2.2 *switch...case*

O `switch` seleciona um entre vários blocos de código com base no valor de uma expressão inteira ou caractere.

```

1  switch (expressao) {
2      case valor1:
3          // codigo para valor1
4          break;
5      case valor2:
6          // codigo para valor2
7          break;
8      default:
9          // codigo se nenhum caso corresponder
10 }

```

1.2.3 *Operador Ternário*

O operador ternário é uma forma compacta de `if...else`:

```

1  variavel = (condicao) ? valor_se_verdadeiro : valor_se_falso;

```

1.3 Operadores

1.3.1 Operadores Aritméticos

- + (adição), - (subtração), * (multiplicação), / (divisão), % (resto da divisão inteira).

1.3.2 Operadores Lógicos

- && (E lógico), || (OU lógico), ! (NÃO lógico).
- Resultam em 0 (falso) ou 1 (verdadeiro).

1.3.3 Operadores Bit a Bit

- & (E bit a bit), | (OU bit a bit), ^ (OU exclusivo), ~ (complemento), « (deslocamento à esquerda), » (deslocamento à direita).
- Operam sobre os bits individuais de valores inteiros.

2 Exercícios de Programação — if...else

2.1 Exercício 1 — Classificação de Triângulos

Escreva um programa que leia três números inteiros positivos representando os comprimentos dos lados de um triângulo. O programa deve verificar se os três valores podem formar um triângulo e, em caso afirmativo, classificá-lo em:

- **Equilátero**: todos os lados iguais;
- **Isósceles**: exatamente dois lados iguais;
- **Escaleno**: todos os lados diferentes.

2.1.1 Regras

1. Para formar um triângulo, a soma de dois lados quaisquer deve ser **maior** que o terceiro lado (desigualdade triangular).
2. Caso os valores não formem um triângulo, imprima a mensagem "Valores não formam um triângulo".
3. Utilize **apenas** estruturas `if`, `else if` e `else` para as verificações e classificação.
4. Use operadores lógicos (`&&`, `||`) para combinar as condições de validade do triângulo.
5. Para verificar se todos os lados são positivos, utilize o operador lógico `&&`.

2.1.2 Exemplo de Entrada e Saída

```
1 // Entrada:
2 3 4 5
3 // Saída:
4 Triângulo Escaleno
```

```
1 // Entrada:
2 5 5 5
3 // Saída:
4 Triângulo Equilátero
```

```
1 // Entrada:
2 1 2 3
3 // Saída:
4 Valores não formam um triângulo
```

2.1.3 Desafio

Inclua uma verificação adicional: utilize uma operação bit a bit para checar se **pelo menos um dos lados** é um número par. Se essa condição for verdadeira, imprima também a mensagem "Pelo menos um lado é par". Use o operador `&` com a máscara 1 para testar a paridade.

2.1.4 Objetivo

Praticar a estrutura `if...else` com múltiplas condições, operadores lógicos e aritméticos, e introduzir o uso de operadores bit a bit em uma verificação simples.

2.2 Exercício 2 — Ano Bissexto com Bit a Bit

Um ano é bissexto se:

- É divisível por 4, **mas não** por 100, **ou**
- É divisível por 400.

Escreva um programa que leia um ano (inteiro positivo) e determine se ele é bissexto ou não, utilizando **apenas** `if...else`.

2.2.1 Regras

1. Leia o ano com `scanf`.
2. Utilize a operação bit a bit `&` para verificar se o ano é divisível por 4. Lembre-se: um número é divisível por 4 se os dois últimos bits forem 0, ou seja, `(ano & 3) == 0`.
3. Para as demais verificações (divisível por 100 e por 400), utilize o operador aritmético `%` (resto da divisão).
4. Combine as condições com operadores lógicos `&&` e `||`.
5. Se o ano for bissexto, imprima "Ano bissexto". Caso contrário, imprima "Ano não bissexto".

2.2.2 Exemplo de Entrada e Saída

```
1 // Entrada:
2 2024
3 // Saída:
4 Ano bissexto
```

```
1 // Entrada:
2 1900
3 // Saída:
4 Ano não bissexto
```

```
1 // Entrada:
2 2000
3 // Saída:
4 Ano bissexto
```

2.2.3 Desafio

Modifique o programa para também informar se o ano é **par ou ímpar** usando apenas operação bit a bit. Imprima essa informação adicional.

2.2.4 Objetivo

Combinar operadores bit a bit com operadores aritméticos e lógicos dentro de uma estrutura `if...else`, reforçando a diferença entre os tipos de operadores.

2.3 Exercício 3 — Cálculo de Tarifa de Energia

Uma concessionária de energia elétrica calcula a tarifa mensal de acordo com as seguintes faixas de consumo (em kWh):

- Até 100 kWh: R\$ 0,50 por kWh;
- De 101 a 200 kWh: R\$ 0,75 por kWh;
- Acima de 200 kWh: R\$ 1,20 por kWh.

Escreva um programa que leia o consumo em kWh (inteiro) e calcule o valor total da conta. Aplicam-se as seguintes regras adicionais:

1. Se o consumo for **negativo**, imprima "Consumo inválido" e encerre.
2. Se o consumo for **maior que 500 kWh**, acrescente uma taxa extra de 15% sobre o valor calculado.
3. Utilize **apenas** `if...else` para todas as decisões.
4. Para verificar se o consumo pertence a uma faixa, combine condições com operadores lógicos `&&`.

2.3.1 Exemplo de Entrada e Saída

```
1 // Entrada:
2 150
3 // Saída:
4 Valor total: R$ 112.50
```

```
1 // Entrada:
2 300
3 // Saída:
4 Valor total: R$ 414.00
```

```
1 // Entrada:
2 -10
3 // Saída:
4 Consumo inválido
```

2.3.2 Desafio

Adicione uma verificação usando operador bit a bit: se o consumo for **múltiplo de 8**, imprima também a mensagem "Consumo múltiplo de 8". Use `(consumo & 7) == 0` para testar.

2.3.3 Objetivo

Praticar `if...else` com faixas de valores, operadores aritméticos e lógicos, e usar uma operação bit a bit para uma condição adicional.

3 Exercícios de Programação — switch...case

3.1 Exercício 4 — Terminal de Autoatendimento de uma Lanchonete

Uma lanchonete deseja desenvolver um programa para funcionar como um **terminal de autoatendimento**. O cliente deverá escolher um produto a partir de um código numérico.

O programa deverá apresentar o seguinte menu:

```
===== MENU =====  
1 - Hambúrguer ..... R$ 18,00  
2 - X-Salada ..... R$ 22,00  
3 - X-Bacon ..... R$ 25,00  
4 - Batata Frita ..... R$ 12,00  
5 - Refrigerante ..... R$ 7,00
```

O usuário deverá informar:

- código do produto;
- quantidade desejada;
- se deseja adicionar uma sobremesa (1 para sim e 0 para não).

O programa deverá utilizar **switch/case** para identificar o produto escolhido.

3.1.1 Regras

Depois de identificar o produto:

1. Calcule o valor dos produtos de acordo com a quantidade.
2. Caso o cliente escolha sobremesa, acrescente **R\$ 8,00** ao pedido.
3. Se o valor total do pedido ultrapassar **R\$ 100,00**, conceda **10% de desconto**.
4. Caso o valor esteja entre **R\$ 50,00 e R\$ 100,00**, conceda **5% de desconto**.
5. Para pedidos inferiores a R\$ 50,00, não haverá desconto.
6. Se o código não existir, informe "Produto inválido". Nesse caso, não realize o cálculo.
7. Ao final, apresente:
 - produto escolhido;
 - quantidade;
 - valor dos produtos;
 - valor da sobremesa;
 - desconto;
 - valor final do pedido.

3.1.2 Desafio

O **switch/case** deverá ser responsável pela **identificação do produto e seu preço**, enquanto as demais decisões deverão ser realizadas utilizando estruturas **if...else**. Pense em como organizar o programa para que um código inválido não seja utilizado nos cálculos.

3.1.3 Objetivo

Praticar o uso do **switch/case** para seleção de opções, combinado com **if...else** para regras de negócio, além de operações aritméticas e de entrada/saída.

3.2 Exercício 5 — Caixa Eletrônico

Você foi contratado para desenvolver um programa simplificado de **caixa eletrônico**.

O sistema deverá apresentar um menu:

```
===== CAIXA ELETRÔNICO =====  
1 - Consultar saldo  
2 - Depositar  
3 - Sacar  
4 - Verificar limite  
5 - Encerrar
```

O programa deverá começar com um saldo de **R\$ 1.000,00**.

O usuário deverá escolher uma operação utilizando um número de **1 a 5**.

O programa deverá utilizar **switch/case** para determinar qual operação será realizada.

3.2.1 Regras

1 — Consultar saldo

Exiba o saldo atual da conta.

2 — Depositar

Solicite o valor do depósito e atualize o saldo. O depósito somente deverá ser aceito se o valor for positivo.

3 — Sacar

Solicite o valor que o usuário deseja sacar. O saque deverá ser permitido somente se:

- o valor for positivo;
- o valor solicitado não ultrapassar o saldo disponível.

Caso o usuário tente sacar um valor maior que o saldo, apresente:

Saldo insuficiente!

4 — Verificar limite

O limite de saque será calculado como:

$\text{limite} = \text{saldo} \times 0,30$

Informe ao usuário o limite disponível para saque.

5 — Encerrar

Apresente uma mensagem informando que a operação foi encerrada.

Código inválido

Caso o usuário informe qualquer número diferente de 1, 2, 3, 4 ou 5, apresente:

Opção inválida!

3.2.2 *Desafio*

Além do `switch/case`, o programa deverá utilizar:

- entrada de dados;
- saída de dados;
- operações aritméticas;
- operadores relacionais;
- operadores lógicos;
- `if`, `else if` e `else`.

3.2.3 *Desafio extra*

Imagine que o usuário possa realizar **várias operações consecutivas**. Como você modificaria o programa para que o menu continue aparecendo até que ele escolha a opção **5 — Encerrar**? Pense em quais estruturas de repetição poderiam ser utilizadas juntamente com o `switch/case`.

3.2.4 *Objetivo*

Entender a diferença entre `switch/case` (escolha de operação) e `if/else` (decisões internas), além de praticar operações aritméticas e interação com o usuário.

3.3 Exercício 6 — Calculadora de Operações Bit a Bit

Escreva um programa que funcione como uma calculadora de operações bit a bit. O programa deve ler dois números inteiros e um código de operação (1 a 5) e, usando `switch/case`, executar a operação correspondente e imprimir o resultado.

As operações são:

- 1 - E bit a bit (`&`)
- 2 - OU bit a bit (`|`)
- 3 - OU exclusivo (`^`)
- 4 - Deslocamento à esquerda (`<<`)
- 5 - Deslocamento à direita (`>>`)

3.3.1 *Regras*

1. Leia dois inteiros `a` e `b` e um inteiro `op` (1 a 5).
2. Utilize `switch` para selecionar a operação.
3. Para as operações de deslocamento, o segundo número `b` representa a quantidade de bits a deslocar.
4. Imprima o resultado em **decimal** (usando `%d`) e em **hexadecimal** (usando `%x`).
5. Se o código da operação for inválido, imprima "Operação inválida".
6. Para o deslocamento à esquerda, o resultado é `a << b`; para a direita, `a >> b`.

3.3.2 Exemplo de Entrada e Saída

```
1 // Entrada:
2 12 5 1
3 // Saída:
4 4 em decimal, 4 em hexadecimal
```

```
1 // Entrada:
2 12 5 2
3 // Saída:
4 13 em decimal, d em hexadecimal
```

```
1 // Entrada:
2 12 1 4
3 // Saída:
4 24 em decimal, 18 em hexadecimal
```

3.3.3 Desafio

Além do `switch/case`, utilize `if` para verificar se os valores lidos são **não negativos** antes de executar a operação. Se algum for negativo, imprima "Valores não podem ser negativos" e encerre.

3.3.4 Objetivo

Praticar o uso do `switch/case` para seleção de operações bit a bit, reforçando o entendimento dos operadores `&`, `|`, `^`, `«` e `»`, e a conversão de saída entre bases decimal e hexadecimal.